Existing workflow term	Agentic equivalent
Inbox, form, spreadsheet, ticket, document	Input channel
Human reads and interprets	Model reasoning / classification / extraction
Human checks policy or previous cases	Retrieval / knowledge source
Human copies data between systems	Tool calls / API actions / RPA if necessary
Human asks colleague or escalates	Handoff / human-in-the-loop
Human completes template or updates record	Structured output / action
Manager checks awkward cases	Approval gate / guardrail
Audit trail in emails/spreadsheets	Trace / transcript / event log

The “black box upgrade” phrase also helps with lock-in. You can say:

> **“If we define the black box in terms of inputs, outputs, rules, controls and evaluation criteria, the product choice becomes replaceable. Copilot Studio, OpenAI Agents SDK, Azure AI Foundry, Power Automate, or a bespoke API are all candidate**

And this is the killer line:

> **“The current implementation is not ‘manual’; it is already an implementation. It just happens to be implemented in people, habits, spreadsheets, email and browser tabs.”**

That sounds slightly provocative, but in a useful way. It makes the existing work analysable as a system rather than hand-wavy “admin”.

Great. Thinking now about a portfolio piece to demonstrate this approach and understanding. To what extent can I, from home, produce a "copilot" version of the same agent I can with the OpenAI SDK? I haven't gone looking on Microsoft properties yet as I still find the Copilot branding mess makes navigation difficult!

I'll treat this as a practical "what can I actually build from home?" question, not a branding tour. I'll check the current Microsoft options and map them to a portfolio strategy.

Thought for 4s >

Yes, but with two important qualifications:

1. **You can produce a convincing Microsoft/Copilot-style portfolio piece from home.**
2. **You may not be able to publish a full enterprise Copilot experience without a suitable Microsoft work/school tenant, Azure subscription, and licensing.**

For a portfolio piece, that is fine. You do not need to prove that you can navigate an employer's tenant governance to demonstrate the architecture.

The practical answer

You can build the same underlying agentic capability in three layers:

```
Reusable agent core
    ↓
OpenAI Agents SDK implementation
    ↓
Microsoft-facing implementation / wrapper / demo
```

The trick is **not** to make "the Copilot" the core asset. Make the core asset the black-box upgrade:

```
Known inputs → agentic processing → known outputs
```

Then show that the same capability can be surfaced through different implementations.

What you can realistically do from home

Option 1: Copilot Studio trial — good for screenshots and terminology

Microsoft says individuals can sign up for a **Copilot Studio trial**, create agents, and test them in the test chat panel. However, the trial does **not** let you publish the agent. Microsoft also notes that rejected sign-up can happen if you use a personal email address; a work or school account may be needed, and organisational admins can block self-service sign-up. Microsoft Learn +1

So this is useful for:

- demonstrating you understand Copilot Studio concepts;
- making a simple version of the same agent;

- capturing screenshots of agent instructions, knowledge, tools/actions, topics, and test chat;
- mapping the Microsoft vocabulary against your OpenAI SDK version.

It is less useful if you want a fully public demo link.

Option 2: Microsoft 365 Agents SDK — better for a developer portfolio

The **Microsoft 365 Agents SDK** is Microsoft's pro-code framework for conversational agents. Microsoft describes it as the "plumbing layer" between a user sending a message in Teams, a website, Slack, or another channel, and the logic you built to respond. Microsoft Learn

That sounds much closer to your portfolio aim because you can keep the real logic in your own code. Microsoft's custom engine agent documentation says custom engine agents let developers bring their own orchestration and AI services, with full control over workflows, models, and integrations. Microsoft Learn

This is the route I would favour for a serious portfolio piece.

Option 3: Custom engine agent / thin proxy — ideal architectural story

Microsoft's custom engine agent model is explicitly for scenarios requiring complex workflows, orchestration, or specific language models; it may require your own hosting for models and orchestrators. Microsoft Learn

That is exactly the architecture you have been describing:

```
Microsoft surface
    ↓
thin agent/proxy/channel adapter
    ↓
your reusable agent backend
    ↓
tools, APIs, evaluation, traces, audit
```

There is also a Microsoft Tech Community example framing this almost exactly: if you already have a multi-agent AI application, you can surface it inside Microsoft 365 Copilot without adding another orchestration layer, using a thin proxy agent built with the Microsoft 365 Agents SDK. TECHCOMMUNI...

That is the phrase I would build the portfolio around: **thin proxy agent**.

Recommended portfolio architecture

I'd build something like this:

```
/example-process
    /docs
        process-overview.md
        current-human-workflow.md
        target-black-box-contract.md
        microsoft-copilot-mapping.md
        evaluation-cases.md

    /agent-core
        domain model
        tool contracts
        orchestration
        tests
        evaluation runner

    /openai-agents-sdk
        Agent + Runner + tools implementation

    /microsoft-copilot-version
        Copilot Studio notes/screenshots OR
        Microsoft 365 Agents SDK wrapper OR
        manifest/config/proxy demo

    /api
        reusable backend endpoints

    /samples
        input emails/forms/docs
        expected outputs
        traces/logs
```

The key portfolio claim would be:

> **"The agent capability is not locked into one product. The same black-box contract is implemented once as an OpenAI Agents SDK workflow and once as a Microsoft-facing agent surface."**

That is much stronger than "I made a Copilot Studio bot."

What the demo should show

Pick a mundane clerical workflow. For example:

```
Input:
An email/request/document containing semi-structured information.

Human baseline:
Read it, classify it, extract fields, check policy, draft response, update record.
```

```
Agentic target:
Classify → extract → validate → ask for missing info → produce structured output → dra
```

Then produce:

Portfolio artefact	Purpose
Black-box contract	Inputs, outputs, rules, exceptions
Human baseline workflow	Shows you understand the existing process
Agent design	Agent, tools, handoffs, guardrails, memory, tracing
OpenAI SDK implementation	Code-first orchestration
Microsoft mapping	Copilot Studio / M365 Agents SDK equivalent
Evaluation set	Shows engineering maturity, not just prompting
Trace/audit examples	Shows governance awareness
Short video/demo GIF	Makes it recruiter/interviewer-friendly

How close can the Copilot version be?

Conceptually: **very close**.

Technically: **it depends which Microsoft route you use**.

Copilot Studio version

Good mapping:

OpenAI SDK	Copilot Studio
Agent instructions	Agent instructions
Tools	Tools/actions/connectors/flows
Knowledge retrieval	Knowledge sources
Handoffs	Connected agents / multi-agent orchestration
Guardrails	Instructions, tool constraints, governance, DLP
Tracing	Activity map, transcripts, analytics

OpenAI SDK	Copilot Studio
Runner	Generative orchestration

But Copilot Studio may make some things more platform-shaped. It is good for demonstrating Microsoft vocabulary, less ideal for showing clean code architecture.

Microsoft 365 Agents SDK / custom engine version

This can be much closer to your OpenAI SDK implementation because your backend remains yours. You can use your own orchestration, your own model calls, your own tools, your own logging, and simply expose it through a Microsoft-compatible conversational surface. Microsoft's docs explicitly distinguish custom engine agents from declarative agents on this basis: custom engine agents give more control over orchestration, AI models, and integrations. Microsoft Learn

That is the version that best supports your "implementation details are the last decision" thesis.

Suggested portfolio framing

I'd title the project something like:

> **Portable Agentic Workflow: upgrading a human-operated clerical process without locking into a platform**

And the headline architecture:

> **One business capability, two implementations: OpenAI Agents SDK and Microsoft-compatible agent surface.**

Then in the README:

> "The current process is treated as a black box with stable interfaces: known inputs, known outputs, rules, exceptions and audit needs. The implementation is then replaced with an agentic workflow. The core logic is kept portable; Microsoft Copilot, Teams, web chat, or another front end can be treated as channels rather than the architecture itself."

That is exactly the kind of thing a founder, technical lead, or AI-forward delivery manager would understand.

My recommendation

For maximum portfolio value, do **not** start in Copilot Studio.

Start with:

1. **Black-box workflow definition**
2. **OpenAI Agents SDK implementation**
3. **Evaluation/test harness**
4. **Thin API boundary**
5. **Microsoft mapping document**
6. Then optionally add either:
    - a **Copilot Studio trial mock-up**, or
    - a **Microsoft 365 Agents SDK wrapper**, if the setup is not too irritating.

That gives you a credible answer in interview:

> "I deliberately kept the core implementation independent. Copilot Studio is one possible deployment surface. If the organisation wants Microsoft governance and Teams integration, I can map the same capability into that environment. But I would not bury the business logic in the canvas unless that was a conscious architectural choice."

Sources

> OK. Let's make up a workflow with enough moving parts to be interesting/familiar. I'm thinking of something like an email requesting a custom report/data extract for a presentation. They've tried various supposedly user-facing business intelligence tools over the years, but every department ends up hiring someone specifically to use them. A new email address just for requesting sales data will be created and unstructured requests can be sent to it. The process will reply requesting more information if required and ultimately produce a file meeting the request. Too complex?

Thought for a second

Not too complex — it is actually an excellent portfolio workflow **provided you keep the first version deliberately narrow**.

It has all the right ingredients:

```
Unstructured email request
        ↓
interpretation / clarification
```

```
        ↓
permission and policy checks
        ↓
data selection
        ↓
report generation
        ↓
file output
        ↓
audit trail and reply
```

That is exactly the sort of “human operating a computer” workflow that agents are good at improving.

The key is not to build “general BI by email.” That would be too complex. Build a **thin, realistic vertical slice**:

> **“A sales-data request agent that accepts natural-language email requests, clarifies missing parameters, validates the request against policy, generates a CSV/XLSX extract from a sample sales dataset, and replies with the finished file or a reason it cannot proceed.”**

That is manageable and familiar.

Why it is a good portfolio case

It demonstrates the black-box upgrade idea beautifully.

Current human process:

```
Requester emails analyst:
"Can I get sales by region for Q3, split by product line, for my board deck?"

Analyst reads request
Analyst asks follow-up questions
Analyst checks whether requester is allowed to see the data
Analyst opens BI/export tools
Analyst filters, groups, checks, exports
Analyst emails spreadsheet back
```

Target agentic process:

```
Requester emails sales-data@example.com

Agent:
- understands the request
- extracts report parameters
- asks clarifying questions if needed
```

```
- checks policy/permissions
- queries dataset
- generates file
- sends response
- logs the decision path
```

Same input surface: email.
Same output surface: file attached to an email.
Different internal implementation.

That is a very clean story.

Keep the first version intentionally bounded

Define a fake but realistic dataset:

```
SalesOrders
- order_id
- order_date
- region
- country
- customer_segment
- product_category
- product_name
- salesperson
- channel
- revenue
- gross_margin
- units
```

Then support only a small set of report shapes at first:

Report type	Example
Sales summary	Revenue and margin by region
Time trend	Monthly sales for a period
Breakdown	Product category by region
Top N	Top 10 customers/products by revenue
Filtered extract	Raw rows for an approved scope

This is enough to feel real without becoming an entire BI platform.

The agent workflow

I'd model it like this:

```
Incoming email
   ↓
Request intake agent
   ↓
Parameter extraction
   ↓
Completeness check
   ↓
Permission / policy check
   ↓
Clarification email OR report plan
   ↓
Data query tool
   ↓
Report file generator
   ↓
Quality check
   ↓
Reply email + attachment
   ↓
Audit log
```

The interesting part is the distinction between **understanding** and **execution**.

The LLM should not directly "make up" the report. It should translate the email into a structured report request:

JSON

```json
{
  "report_type": "sales_summary",
  "metrics": ["revenue", "gross_margin"],
  "dimensions": ["region", "product_category"],
  "date_range": {
    "start": "2025-07-01",
    "end": "2025-09-30"
  },
  "filters": {
    "country": "UK"
  },
  "output_format": "xlsx",
  "purpose": "board presentation"
}
```

Then deterministic code generates the file from known data.

That is a strong engineering message: **LLM for interpretation; code for calculation.**

Clarification behaviour

This is where the demo becomes agentic rather than just “prompt in, spreadsheet out.”

Example incoming email:

> “Can you send me sales for the last quarter by region? Need it for Thursday’s presentation.”

The agent can infer:

```
metric: probably revenue
period: last quarter, but relative to current date
dimension: region
format: unspecified
```

It might reply:

> “I can prepare that. Please confirm whether you want revenue only, or revenue and margin, and whether Excel format is suitable.”

A better request:

> “Please send Q4 2025 UK revenue and gross margin by region and product category as Excel.”

That can proceed without clarification.

Policy and permission checks

This gives the portfolio piece corporate realism.

Examples:

Policy	Behaviour
User may only request their own region	Reject or narrow request
Customer-level data requires approval	Ask for manager approval
Board presentation permits aggregated data only	Generate grouped summary, not raw rows
Margin data restricted to senior users	Remove margin or request approval
Date range over 24 months too large	Ask for narrower scope

This is useful because it shows that agents are not just “chatty automation.” They operate inside controls.

Outputs

For a portfolio demo, generate:

1. **Clarification email**
2. **Report plan**
3. **SQL query or pandas operation**
4. **CSV/XLSX file**
5. **Executive summary**
6. **Audit log**

For example, the reply might say:

```
Attached is the requested Q4 2025 UK sales summary by region and product category.

Included metrics:
- Revenue
- Gross margin
- Units sold

Filters applied:
- Country: UK
- Order dates: 2025-10-01 to 2025-12-31

No customer-level data is included.
```

The file itself can be a generated `.xlsx` with tabs:

```
Summary
Data
Request Metadata
```

That last tab is a nice touch. It makes the governance/audit story visible.

Suggested agent design

For OpenAI Agents SDK, you could split it into specialists, but do not overdo it.

A good first version:

```
Triage / Intake Agent
- understands the email
- extracts structured intent
- decides whether clarification is needed

Policy Agent
- checks requester, permissions, and sensitivity

Report Builder Agent
```

```
- converts approved request into query plan
- calls data/report generation tools

Response Agent
- drafts the email response
- explains assumptions, filters, and exclusions
```

Or simpler:

```
One Agent
- with tools:
  - parse_request
  - check_policy
  - query_sales_data
  - generate_excel
  - draft_reply
  - log_audit_event
```

For a portfolio piece, I would actually start with **one agent plus tools**, then include a README section saying how it could evolve into multiple agents. That avoids the common "multi-agent for its own sake" smell.

Microsoft/Copilot mapping

This maps well to Microsoft terminology too:

Your workflow concept	Copilot/Microsoft equivalent
Sales data mailbox	Email/channel trigger, Power Automate, Graph connector
Request understanding	Copilot Studio agent instructions / generative orchestration
Missing information	Topic/question flow or agent clarification
Data access	Connector, API tool, Dataverse, SQL, Fabric, Power BI semantic model
Policy check	Tool/action plus DLP/security/governance
File generation	Power Automate, Azure Function, API tool
Human approval	Power Automate approvals
Reply with attachment	Outlook connector / Graph API

Your workflow concept	Copilot/Microsoft equivalent
Audit log	Dataverse, SharePoint list, Application Insights, custom DB

This would let you say:

> "The OpenAI implementation uses Agents SDK tools and structured outputs. The Microsoft implementation would expose the same core operations as tools/actions, with Outlook/Graph as the channel, Power Automate for approval where needed, and Copilot Studio or a custom engine agent as the conversational layer."

That is precisely the portability story.

Sensible MVP boundary

I'd make the MVP support this:

```
Input:
Plain-text email request

Supported data:
Synthetic sales dataset

Supported outputs:
CSV and XLSX

Supported report types:
- sales by region
- sales by product category
- monthly trend
- top N products/customers
- filtered raw extract, subject to permission

Supported controls:
- requester role
- region entitlement
- margin-data entitlement
- customer-level-data restriction

Supported interactions:
- request clarification
- produce report
- reject with explanation
- escalate for approval
```

That is plenty.

What not to build first

Avoid these in v1:

- real email ingestion;
- real Microsoft 365 tenant integration;
- real Power BI embedding;
- arbitrary natural-language SQL over unknown schemas;
- charts and dashboards;
- row-level security beyond a simple mock policy table;
- full conversation memory;
- complex multi-agent choreography.

You can simulate inbound emails as `.eml`, `.txt`, or JSON files. That is perfectly adequate for a portfolio demo.

A good repo narrative

Something like:

> **Email-to-Report Agent: portable agentic workflow for sales-data requests**

README summary:

> "This project demonstrates how an existing human-operated clerical workflow can be upgraded without changing its external interfaces. Requesters send unstructured emails asking for sales reports. The agent extracts intent, asks for clarification where needed, checks policy, generates a structured report from a sample dataset, and drafts a reply with an attached file. The core capability is kept independent of any one agent platform, with mappings for OpenAI Agents SDK and Microsoft Copilot-style deployment."

That is a strong portfolio piece because it shows:

- real business process understanding;
- agentic decomposition;
- structured output;
- deterministic tool use;
- clarification loops;
- governance;
- portability;
- Microsoft awareness without being trapped in Microsoft tooling.

My verdict

Not too complex. It is probably **the right level of complex**.

The only discipline required is to state clearly:

> **This is not a general BI replacement. It is an agentic request-handling workflow for a bounded family of sales-data extracts.**

That keeps it credible and buildable.